

Projektuppgift

[DT211G D2492](#)

Projektuppgift Frontendbaserad Webbutveckling

Rune Granath



Mittuniversitetet
MID SWEDEN UNIVERSITY

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Rune Granath, rugr2500@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: HT, 2025

Sammanfattning

Målet med detta projekt har varit att utveckla en interaktiv webbapplikation, mashupen DineVibe, som tar två stycken olika Application Programming Interfaces(API)-resurser och låter dessa interagera med varandra, med ett extra fokus på animation.

I detta fall är det två intressen, matlagning och musik, som får arbeta tillsammans för att hitta ett tips på maträtt till låten man söker på, och här har Spotify Web API för musikdata och samt Spoonacular API för receptdata använts. Utefter längden på låten får du sedan resultatet av sökningen, exempelvis snabblagade rätter för korta låtar och långkok för längre musikstycken.

Projektet har implementerats med ett fokus på asynkron programmering för anrop och en automatiserad utvecklingsmiljö, vilket huvudsakligen har stötats av byggverktyget Vite.

Slutsatsen är att det finns oändliga möjligheter för att kunna skapa nischade webbtjänster.

Sammanfattning	iii
Terminologi Akronymmer och förkortningar	iv
7 Källförteckning	14

Terminologi Akronymer och förkortningar

Akronymer och förkortningar

- **API** *Application Programming Interface*. Ett gränssnitt som gör det möjligt för olika mjukvaruapplikationer att kommunicera och utbyta data med varandra.
- **DOM** *Document Object Model*. En programmeringsgränssnitt för HTML-dokument som representerar sidans struktur som ett träd av objekt, vilket möjliggör manipulation via JavaScript.
- **ES6+** *ECMAScript 2015* (och senare versioner). Den moderna standarden för JavaScript-språket som introducerade funktioner som pilar, klasser och moduler.
- **JSON** *JavaScript Object Notation*. Ett lättviktigt format för datalagring och kommunikation som är enkelt för både människor att läsa och maskiner att tolka.
- **SCSS/Sass** *Syntactically Awesome Style Sheets*. Ett förbehandlingspråk (preprocessor) för CSS som tillåter variabler, nästling och mer avancerad logik i designarbetet.

Tekniska begrepp och verktyg

- **Asynkron programmering** En programmeringsmetod (ofta via `async/await` i JavaScript) som tillåter att tidskrävande processer, såsom API-anrop, körs i bakgrunden utan att frysa programmets övriga exekvering.
- **GitHub Actions** Ett verktyg för automatisering av arbetsflöden, i detta projekt använd för *Continuous Deployment* (CD) för att automatiskt bygga och publicera webbplatsen.
- **JSDoc** Ett märkspråk som används för att kommentera JavaScript-kod, vilket möjliggör automatisk generering av teknisk dokumentation.
- **Mashup** En webbapplikation som kombinerar data eller funktionalitet från två eller flera externa källor för att skapa en ny tjänst.
- **Vite** Ett modernt byggverktyg och utvecklingsserver som optimerar laddningstider och paketering av källkod under utvecklingsprocessen.

1 Introduktion

Detta inledande kapitel ger en övergripande bild av projektet DineVibe. Här presenteras bakgrunden till varför applikationen skapats, vilka problem den ämnar lösa samt vilka mål och avgränsningar som funnits under utvecklingsprocessen.

1.1 Bakgrund och problemmotivering

Idén till DineVibe föddes ur behovet av att förenkla vardagsbeslut genom att kombinera nytta med nöje. Många upplever beslutsångest och monotoni kring vad de ska laga till middag, samtidigt som musik är en central del av deras vardag och ofta kan visa hur man känner för stunden. Genom att föra dessa samman skulle det kunna fylla den processen med mer lust och matcha valet av mat till mående för ett mer kreativt beslut. Personer som kan tänkas använda appen är mest troligt yngre användare med god vana för nya sorters appar, vilket då kräver ett modernt utseende och modern design.

Problemet som ska lösas är alltså att automatisera beslutsfattandet, medan tekniskt sett utmaningen ligger i att få två helt olika API-tjänster, Spotify och Spoonacular, att utbyta information på ett logiskt sätt.

1.2 Övergripande syfte / Högnivåproblemformulering

Projektets syfte är att utveckla en fungerande webbapplikation som demonstrerar färdigheter i att hämta, transformera och presentera data från externa JSON-källor. Vidare syftar projektet till att använda en professionell och automatiserad arbetsprocess med moderna verktyg som Vite och GitHub Actions.

1.3 Avgränsningar

För att hålla projektet inom tidsramen har följande avgränsningar gjorts:

- Inga ramverk (som React eller Angular) används.
- Systemet hanterar inte användarinloggning eller har en större nivå av säkerhetstänk kring API-nycklar.
- Sökningen utgår från låttitlar och returnerar en låt från Spotify samt ett recept från Spoonacular. Av låtens egenskaper används främst dess speltid som underlag för receptmatchningen.

1.4 Konkreta och verifierbara mål / Detaljerad problemformulering

Målet med uppgiften har främst varit att två eller fler API-tjänster ska kunna kommunicera med varandra, i en automatiserad utvecklingsmiljö där resultatet ska vara en app-liknande sida som även ska innehålla olika genomtänkta animationer.

2 Teori / Bakgrundsmaterial

Detta kapitel definierar de tekniska begrepp och verktyg som ligger till grund för projektets genomförande. En förståelse för dessa tekniker är nödvändig för att kunna tolka applikationens arkitektur och funktionalitet.

2.1 JSON-baserade webbtjänster och API

Ett *Application Programming Interface* (API) fungerar som ett standardiserat gränssnitt som möjliggör kommunikation mellan två oberoende mjukvarusystem. I detta projekt används REST-baserade API:er som levererar data i formatet *JavaScript Object Notation* (JSON). JSON är ett textbaserat format för datalagring som bygger på nyckel-värde-par, vilket gör det enkelt för JavaScript att parsas (tolka) till objekt.

Genom att använda JSON kan DineVibe hämta strukturerad information från externa servrar, såsom låttitlar från Spotify [1] eller ingredienslistor från Spoonacular [2], och integrera dessa i det lokala användargränssnittet.

2.2 Asynkron programmering och Fetch API

För att skapa en modern och följsam webbapplikation används asynkron programmering. Traditionell synkron programmering exekverar kod rad för rad, vilket innebär att webbläsaren "fryser" medan den väntar på svar från en extern server. Genom att använda *fetch-metoden* och nyckelorden *async* och *await* kan anropen ske asynkront [3]. Detta innebär att applikationen fortsätter att köras medan data hämtas i bakgrunden, vilket resulterar i en oavbruten användarupplevelse.

2.3 Mashup-arkitektur

Begreppet "mashup" avser en webbapplikation som kombinerar innehåll från mer än en källa för att skapa en ny, integrerad tjänst. DineVibes arkitektur utvecklades till att skapa en logisk koppling mellan de två API-tjänsterna, där låtens speltid användes för att beräkna en längdfaktor som i sin tur styrde både receptkategori och maximal tillagningstid.

2.4 CSS-animeringar och användarupplevelse (UX)

Animeringar i ett gränssnitt handlar inte enbart om estetik, utan fungerar som visuell feedback till användaren. Genom att använda CSS-transitions och keyframes kan applikationen kommunicera statusförändringar, såsom

att data laddas eller att ett resultat har hittats. I projektet används animeringar även för att para ihop genomsnittliga åtlängder med recept samt för att ge en introduktion för användaren med instruktioner till sidan.

2.5 Automatiserade arbetsprocesser med Vite och Node.js samt publicering

Node.js fungerar som körsmiljö för projektets byggverktyg, Vite [4]. Vite används för att automatisera repetitiva uppgifter som tidigare krävde manuell hantering.

- **Module Bundling:** Kombinerar flera JavaScript-filer till en optimerad fil för att minska antalet HTTP-anrop.
- **Transpiling:** Säkerställer att modern JavaScript-kod fungerar även i äldre webbläsare.
- **Minifiering:** Komprimerar källkod genom att ta bort blanksteg och korta ner variabelnamn, vilket sänker sidans laddningstid.

Själva publiceringen konfigurerades sedan och kopplades till GitHub pages och med hjälp av GitHub actions vilket gjorde att 'push' av kod automatiskt triggade bygget och en uppdatering av sidan.

2.6 CSS-preprocessorer (Sass/SCSS)

Sass (*Syntactically Awesome Style Sheets*) är en preprocessor som utökar funktionaliteten i standard-CSS. Genom att använda filändelsen .scss kan utvecklaren använda variabler för färger och typsnitt, nästla selektorer för bättre överblick och dela upp koden i moduler. Detta skapar en mer lättunderhållen och skalbar kodbas.

2.7 Versionshantering och CI/CD

Git används för versionshantering, vilket möjliggör spårning av ändringar i källkoden och samarbete. Projektet tillämpar även *Continuous Integration/Continuous Deployment* (CI/CD) via GitHub Actions. Detta innebär att varje gång ny kod skickas till GitHub, startas en automatiserad process som bygger, optimerar och publicerar webbplatsen på GitHub Pages.

2.8 Designprinciper för mobila webbapplikationer

För att ge DineVibe en autentisk "app-känsla" har designprocessen utgått från etablerade mönster inom mobil gränssnittsdesign (UI). Inspiration har

hämtats via moodboards på tjänster som Pinterest för att identifiera en modern och funktionell estetik, medan matchande färgscheman har letats på Colorhunt.co. Följande principer har tillämpats:

- **Kortbaserad layout (Card-Based):** Istället för traditionella listor presenteras recept och låtar som separata "kort". Detta skapar en ren och modulär struktur som är enkel att skanna visuellt och fungerar väl i responsiva flöden.
- **Glassmorphism:** En modern designstil som använder halvgenomskinliga bakgrunder med suddighetseffekter (*backdrop-filter: blur*). Detta har använts för att skapa ett lyxigt djup i gränssnittet, där elementen tycks sväva ovanför bakgrunden, likt operativsystem som iOS och Windows 11.
- **Dashboard-vy:** Applikationens huvudvy är utformad som en instrumentbräda där data från Spotify (spelaren) och Spoonacular (receptet) presenteras sida vid sida för att tydliggöra kopplingen i denna mashup.

2.9 Begränsningar i externa API-resurser

En kritisk aspekt vid utveckling av mashups är beroendet av tredjepartstjänsters tillgänglighet och dokumentation. Under projektets gång identifierades en teknisk begränsning i Spotifys ekosystem.

Tidigare erbjöd Spotify en endpoint kallad *Audio Features* (för analys av t.ex. "energy" och "danceability"), men i december 2024 begränsades åtkomsten till denna funktion drastiskt. Spotify kräver numera en manuell granskning och godkännande av applikationer för att få tillgång till dessa data. För att säkerställa att projektet förblev funktionellt och skalbart utan dessa begränsningar, valdes en alternativ, enklare logik. Istället för energinivåer baseras matchningen på en kombination av låtens tidslängd (*duration*), vilket är publikt tillgängligt via Spotifys sök-API. Detta för att prioritera applikationens driftsäkerhet och tillgänglighet för slutanvändaren.

3 Metod

Arbetet med DineVibe inleddes med en planeringsfas där fokus låg på att hitta API-resurser som kunde skapa en meningsfull och intressant interaktion. Genom förstudien valdes Spotify Web API och Spoonacular Food API, då båda är någorlunda erkända tjänster som erbjuder välstrukturerade data i JSON-format och en dokumentation som tydligt beskriver hur asynkrona anrop ska utföras.

Efter att de tekniska förutsättningarna fastställts påbörjades designprocessen med ett sökande efter inspiration och trender för design. Med hjälp av moodboards på Pinterest efterforskades moderna designtrender, vilket ledde fram till beslutet att implementera en kortbaserad layout och "Glassmorphism" för att ge webbapplikationen en autentisk och lyxig känsla.

Själva systemutvecklingen genomfördes med en iterativ programmeringsmetod i JavaScript, där fokus låg på att bygga robusta asynkrona funktioner med Fetch API och att arbeta främst med Javascriptfilen för att skapa huvudinnehållet på sidan. AJAX-anrop (Asynchronous JavaScript and XML) användes som den primära metoden för datakommunikation för att fastställa att användargränssnittet blev interaktivt medan data hämtas. Denna metodik möjliggjorde att data kunde hämtas och transformeras i bakgrunden, vilket är avgörande för att skapa den efterfrågade "webb-app"-känslan utan störande sidomladdningar.

En central del av utvecklingsmetoden var skapandet av den logiska bryggan mellan de två API-tjänsterna. Låtens längd hämtades i millisekunder från Spotify, konverterades till minuter och normaliserades till ett värde mellan 0 och 1. Det normaliserade värdet användes sedan både för att välja en receptkategori och för att beräkna en maximal tillagningstid, *maxReadyTime*. Dessa värden användes tillsammans i anropet till Spoonacular för att söka fram ett passande recept.

Genom att använda byggverktyget Vite automatiserades optimeringen av källkoden, såsom minifiering och modulhantering, vilket säkerställde att den publika versionen av webbplatsen alltid var prestandaoptimerad.

För att hantera kodens versionshantering användes Git i kombination med en automatiserad kedja för att bygga upp sidan automatiskt till Github pages via GitHub Actions. Github startar då upp en virtuell dator vid bygget som installerar samma Node.js-miljö som din och fullföljer det.

Slutligen kvalitetssäkrades projektet genom att använda JSDoc för teknisk dokumentation av samtliga funktioner, samt genom tester där olika sökscenarier kördes för att verifiera att algoritmerna för tidsmatchning och de villkorsstyrda animationerna fungerade korrekt i enlighet med projektets uppsatta mål.

4 Konstruktion / Lösningalternativ

Den tekniska konstruktionen av DineVibe vilar på en arkitektur där JavaScript fungerar som motorn som driver interaktionen mellan användargränssnittet och de externa API-resurserna. Vid applikationens start initieras ett sökformulär som fångar upp användarens input och skickar denna vidare till en dedikerad funktion för kommunikation med Spotify Web API. En box med animeringar som förklarar konceptet kort slussar även in användaren till sidan. En *conic-gradient* med en täckande bakgrund för *padding-box*-delen används främst där för att ge intrycket av ett ljus som rör sig kanten av boxens yttre *border box*-del. Med hjälp av variabeln *border-angle* roteras gradienten kontinuerligt, medan gradtalen i koden definierar ljusets spridning samt vilka färger som ska visas längs dess väg.

Genom att använda moderna JavaScript-konstruktioner som *async* och *await* genomförs AJAX-anrop mot Spotifys sök-endpoint, där ett JSON-svar innehållande metadata för låtar returneras.

```
const music = await getMusicMood(searchTerm);
```

Figur 4.1. Konstanten music väntar på funktionen getMusicMood

Konstruktionens hjärta är den transformeringslogik som tar fältet *duration_ms* från Spotifys data och räknar om detta värde från millisekunder till minuter genom division med 60 000. Värdet normaliseras sedan mot ett tak på 5 minuter för att skapa variabeln *lengthScore*, vars värde begränsas till intervallet 0-1. Det normaliserade värdet används sedan för att kategorisera receptsökningen i nästa steg.

```
const minutes = track.duration_ms / 1000 / 60;  
  
let lengthScore = minutes / 5;  
if (lengthScore > 1) lengthScore = 1;
```

Figur 4.2. Beräkning av lengthScore genom konvertering och normalisering av tidsdata.

```
if (music.lengthScore >= 1.0) {  
  foodQuery = "slow cook";  
} else if (music.lengthScore >= 0.8) {  
  foodQuery = "dinner";  
} else if (music.lengthScore >= 0.5) {  
  foodQuery = "pasta";  
} else {  
  foodQuery = "snack";  
}
```

Figur 4.3. Genom denna logiska brygga transformeras musikens egenskaper till en sökterm för recept. Variabeln *foodQuery* tilldelas en kategori beroende på *lengthScore*.

Funktionen *fetchRecipe* tar därefter emot *lengthScore* och använder värdet för att beräkna *maxReadyTime*. Beräkningen omvandlar det normaliserade värdet till en maximal tillagningstid inom intervallet 15–90 minuter. *maxReadyTime* skickas sedan som parameter i anropet till Spoonacular och begränsar sökresultatet till recept vars tillagningstid inte överstiger den beräknade gränsen.

För att optimera prestandan och minska antalet onödiga anrop har konstruktionen försetts med felhantering som validerar att en låt faktiskt hittats innan sökningen efter recept påbörjas.

Det visuella gränssnittet är uppbyggt med en kortbaserad layout där varje recept presenteras i ett dynamiskt genererat HTML-element. En introduktionsbox med tillhörande animeringar och en kort instruktion välkomnar användarna. För att skapa en levande och responsiv upplevelse har CSS-animeringar integrerats direkt i logiken; genom att använda villkorsstyrda klasser i JavaScript triggas olika visuella effekter beroende på resultatet av sökningen. Exempelvis har en specifik CSS-animation med "dansande sniglar" konstruerats för att visas vid recept med lång tillagningstid:

```
if (isLongest) {
  const snailsContainer = document.createElement("div");
  snailsContainer.classList.add("snails-container");

  for (let i = 0; i < 6; i++) {
    const snailEl = document.createElement("div");
    snailEl.classList.add("dancing-snail");
    snailEl.textContent = "🐌";
    snailsContainer.appendChild(snailEl);
  }

  document.body.appendChild(snailsContainer);
}
```

Figur 4.4. I Javascript: de längsta låtarna triggar en animation där sniglar läggs till i en for-loop.

Med hjälp av *@keyframes* och *transform: scaleX(1)* och *scaleX(-1)* i CSS "vänder" de sig i en tidslinje.

Rent stilistiskt är applikationen konstruerad med SCSS, vilket har möjliggjort användandet av avancerade tekniker som *backdrop filter: blur* för att uppnå den tänkta Glassmorphism-effekten.

```
@mixin glass-effect($blur: 10px, $bg: rgba(255, 255, 255, 0.05)) {
  background-color: $bg;
  -webkit-backdrop-filter: blur($blur);
  backdrop-filter: blur($blur);
}
```

Figur 4.5. Backdrop-filter fungerar bara på det som finns bakom element. 10px är graden av suddighet och 0.05-värdet är opaciteten.

För att separera konfiguration från den funktionella logiken har API-nycklar och konfigurationsvärden brutits ut från huvudkoden. Lösningen förbättrar kodens struktur och underhållbarhet, men innebär inte att API-nycklar som används direkt i klienten är skyddade från insyn.

När det var dags att publicera till Github pages med Github actions skapades en .yml-fil med olika instruktioner, däribland npm run build den viktigaste, för att kunna automatisera bygget och publiceringen med en enda kod-'push' till Github.

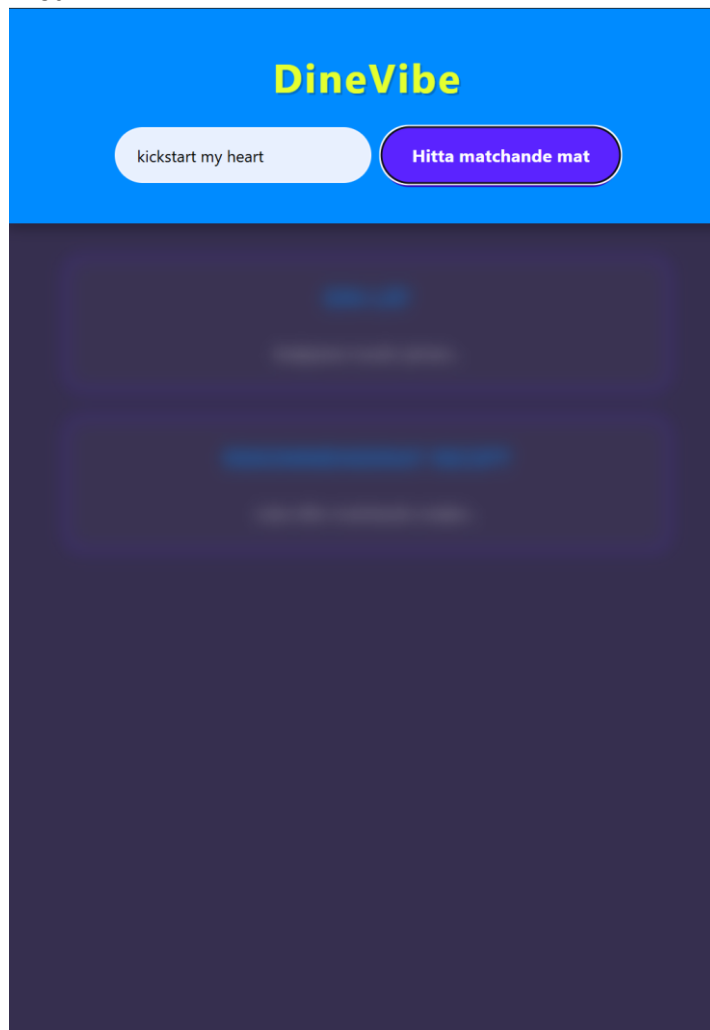
Slutligen har hela konstruktionen paketerats och optimerats med hjälp av byggverktyget Vite, som vid varje publicering minifierar JavaScript-filerna och kompilerar SCSS-koden till standard-CSS för maximal kompatibilitet och snabba laddningstider i webbläsaren.

5 Resultat

I detta kapitel presenteras den färdiga webbappen och hur de tekniska lösningarna som beskrivits i kapitel 4 fungerar i praktiken för slutanvändaren.

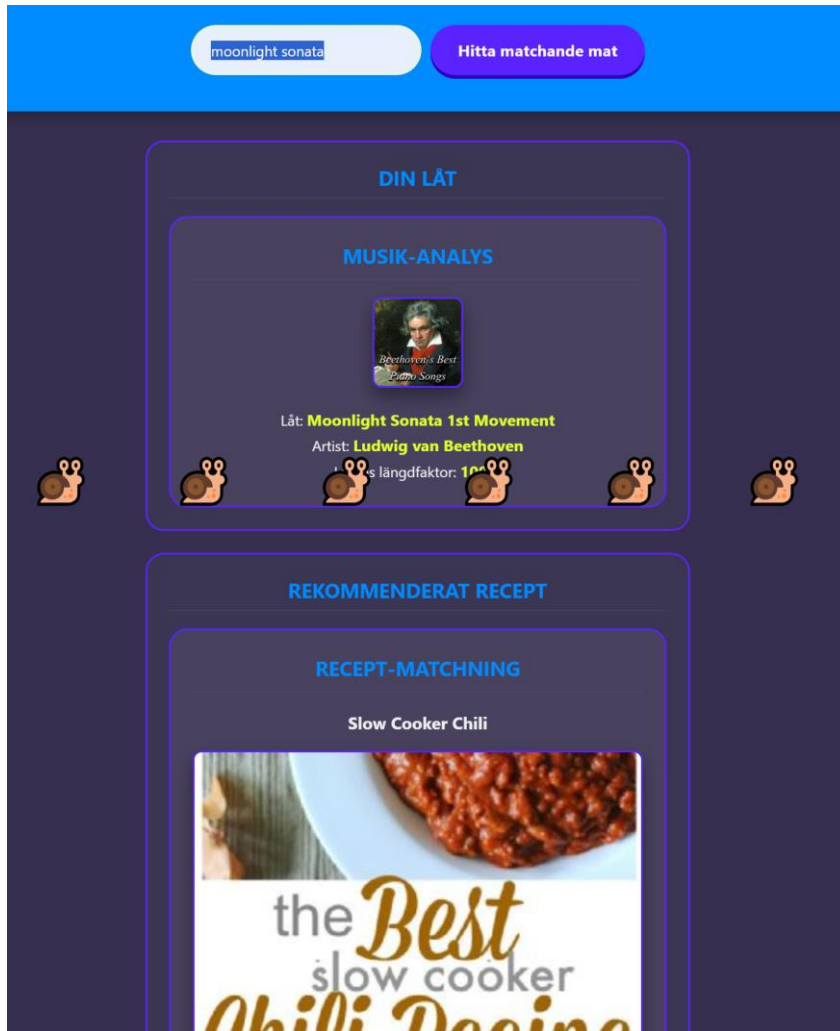
För att säkerställa att integrationen mellan Spotify API och Spoonacular API fungerar korrekt testades systemet med olika typer av musik. Resultaten visar att låtinformationen hämtas och presenteras korrekt med artistnamn och låtlängd. Receptmatchningen visar samtidigt ett recept med tillagningstid, en tillhörande text som beskriver kopplingen till låtlängden samt en länk till receptet. Testerna visar därmed att data från Spotify kan bearbetas och användas som underlag för receptsökningen i Spoonacular. Tester med låtar av olika längd visade även att olika värden på *lengthScore* ledde till olika receptkategorier och maximala tillagningstider.

Appen har ett enhetligt gränssnitt som följer den planerade designstrategin med "Glassmorphism":



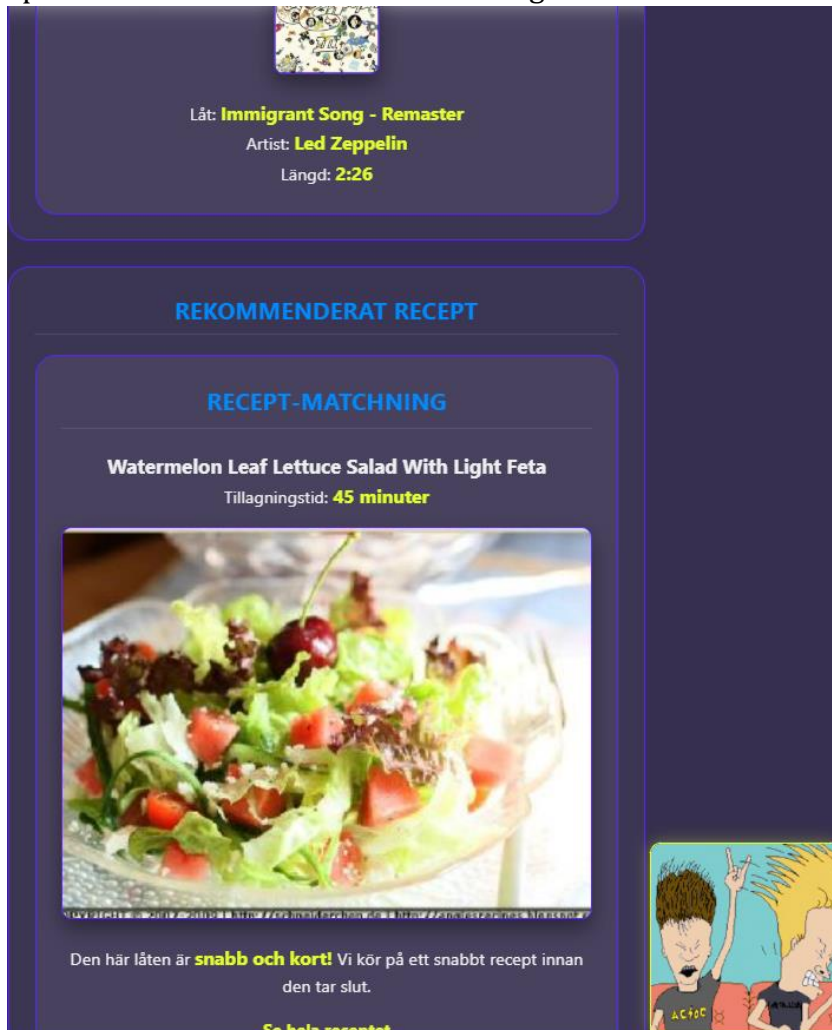
Figur 5.1. Den tvingade laddningssekvensens utseende efter en sökning har gjorts.

En sökning på en lång låt visar ett grytrecept och triggas de dansande sniglarna över skärmen:



Figur 5.2. Låtar där lengthScore är max

Och de kortaste låtarna ger oss ett snabbt recept och triggas visningen av en specifik animation i gränssnittets nedre hörn.



Figur 5.3. Alla låtar med lengthScore under 0.5 får en egen animation

6 Diskussion

Sammantaget uppfyller applikationen projektets huvudsakliga mål genom att kombinera data från två externa API-tjänster i ett gemensamt gränssnitt.

Efter upptäckten att Spotify begränsat den data man får från deras API blev beslutet att simplificera resultatet användaren fick ut från sin sökning något trist, men det var också nödvändigt för att man över huvud taget skulle kunna använda tjänsterna tillsammans. Om utvecklandet hade fortsatt i två veckor till hade fokus legat främst på implementering av mer data från Spoonaculars API, till exempel vinmatchning eller ingredienslistor där sökresultaten hade varit mer obegränsade och för att få variation och liv i användandet av tjänsten.

Under testningen upptäcktes även att den ursprungliga söktermen *slow cook* gav få eller inga resultat i kombination med den beräknade maximala tillagningstiden, och ändrades därför till *stew*. Detta visar en begränsning med att kombinera två externa API-tjänster: även om den interna logiken fungerar kan resultatet påverkas av hur den externa tjänsten tolkar en viss sökterm.

Ett annat problem som upptäcktes var att samma recept kom tillbaka flera gånger. Det fick den enkla lösningen med att slumpa sökresultatet i API-anropet.

Annars har arbetet, främst med automatisering, visat prov på hur man faktiskt kommer börja jobba "på riktigt" som en utvecklare, och allt krångel med diverse inställningar för att få det på plats har gett en bra grund och förståelse för att sedan kunna gå vidare till mer hela lösningar, även om de säkerligen kommer med sina egna krångel.

7 Källförteckning

- [1] **Spotify for Developers.** Web API Reference.
<https://developer.spotify.com/documentation/web-api> (Hämtad 2026-03-25).
- [2] **Spoonacular API.** Documentation. <https://spoonacular.com/food-api/docs> (Hämtad 2026-03-25).
- [3] **MDN Web Docs.** Using the Fetch API.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (Hämtad 2026-03-25).
- [4] **Vite.** Next Generation Frontend Tooling. <https://vitejs.dev/guide/> (Hämtad 2026-03-25).